

QA Automation Agent

Comprehensive Project Report

AI-Powered Browser Automation Platform using Local LLMs

Field	Value
Report Date	June 12, 2026
Prepared For	venkat.sabapathi@promon.co.in
Project Location	C:\Users\Sandhosh\Downloads\QA-Automation-Agent
Repository Branch	main (clean working tree)
Application Version	2.0.0 (backend), 1.0.0 (frontend)

1. Executive Summary

QA Automation Agent is a full-stack, self-hosted QA automation platform that converts natural-language test flow descriptions into executable browser automation. A FastAPI backend orchestrates local LLMs (served by Ollama) to plan test steps, drives a Chromium browser via Playwright to execute them, and produces artifacts such as screenshots, execution logs, and PDF test reports. A Next.js 15 / React 19 frontend provides a dashboard for authoring flows, running single or bulk executions, watching live browser frames over WebSockets, and reviewing results.

The standout characteristic of the project is that it requires no cloud AI services: all inference runs locally through Ollama (a text model for flow planning and a vision model for screen understanding), making it suitable for testing internal or sensitive applications.

2. Purpose and Key Capabilities

- **Natural-language test authoring** - testers write flows as plain-English step lists (e.g. "Click the Log in button", "Enter Username in input box \${CSR_USER}"); an LLM converts them into a structured JSON action plan (navigate, click, type, select, wait, verify, scroll, press_key, screenshot).
- **Local LLM inference via Ollama** - a text model (default qwen2.5:14b) parses flows, and a vision model (default qwen2.5vl) interprets screenshots when DOM-based targeting is insufficient.
- **Playwright browser execution** - headless or headed Chromium, with Windows-specific ProactorEventLoop threading to support subprocesses, automatic relaunch recovery, and shared browser/login sessions across sequential bulk runs.
- **Variable substitution and secrets handling** - \${VAR} placeholders in flow steps are resolved from per-flow credentials, process environment, or a live-reloaded .env file, so credentials never need to be hard-coded in flows.
- **Run-time value capture** - steps can copy values from pages (e.g. an ICCID from a table) and reuse them later in the same run via \${VAR} substitution.
- **Live execution monitoring** - base64 JPEG browser frames and step logs are streamed to the frontend over WebSockets during execution.
- **Bulk execution** - multiple flows can be queued and run sequentially with session reuse; a dedicated bulk results page summarizes outcomes.
- **Artifacts and reporting** - screenshots (configurable mode: fail_only, final, all), per-execution logs, generated PDF test reports (ReportLab), and a flow "memory" store that records failures to inform future runs.
- **Authentication** - JWT-based platform login (python-jose + passlib/bcrypt) with a 24-hour token lifetime, separate from credentials used inside test flows.

3. System Architecture

The system is a classic two-tier web application with a local AI sidecar. The frontend calls the backend REST API (prefix /api) over HTTP and subscribes to WebSocket channels for live execution data. The backend talks to Ollama over its HTTP API (default http://localhost:11434) and controls Chromium through Playwright.

High-level flow of a test execution:

- **Step 1.** User authenticates (admin login) and selects or authors a flow (UI or JSON file on disk).
- **Step 2.** Orchestrator reloads .env, substitutes \${VAR} credentials/URLs into steps, and asks the LLM client to convert natural-language steps into a structured JSON plan (with a rule-based smart parser as fallback if the LLM is unavailable).

- **Step 3.** BrowserAgent launches Chromium (in a dedicated ProactorEventLoop thread on Windows) and executes each action, streaming logs and JPEG frames back through async callbacks.
- **Step 4.** Vision model is consulted for screen understanding where needed; failures are recorded into the per-flow memory store.
- **Step 5.** Artifacts are persisted (screenshots, execution record); a PDF report is generated via pdf_reporter; results are surfaced in the frontend (execution detail, logs, reports pages).

3.1 Technology Stack

Layer	Technology	Notes
Backend framework	FastAPI 0.115.6 + Uvicorn 0.32.1	Async REST API, WebSockets, static file serving for screenshots
Browser automation	Playwright 1.49.1 (Chromium)	Headless by default; slow-mo and navigation timeout configurable
AI runtime	Ollama (local)	Text model: qwen2.5:14b; Vision model: qwen2.5vl (config defaults)
Auth/security	python-jose, passlib[bcrypt]	JWT access tokens, 24h expiry
PDF generation	ReportLab >= 4.2.0	Execution test reports
Data validation	Pydantic 2.10.4	Schemas for flows, executions, logs
HTTP client	httpx 0.28.1	Async calls to Ollama
Frontend framework	Next.js 15.1.3 + React 19	App Router, TypeScript 5.7
UI styling	Tailwind CSS 3.4 + Radix UI + lucide-react	Dialog, tabs, tooltip, dropdown primitives; CVA + tailwind-merge
Frontend HTTP	axios 1.7.9	REST client in lib/api.ts
Testing	pytest 8.3.4 + pytest-asyncio	Backend test tooling (declared)
Runtime requirements	Python 3.10+, Node.js 18+	Per README

4. Backend Components (backend/app)

4.1 Agents (the core engine)

Module	Size	Responsibility
agents/browser_agent.py	~130 KB	The largest module. Playwright-driven executor: launches Chromium, performs click/type/select/navigate/etc., captures screenshots per configured mode, streams JPEG frames, tracks captured variables, skips redundant logins per authenticated domain, supports shared browser/context for bulk sequential runs, and handles Windows ProactorEventLoop thread dispatch.
agents/llm_client.py	~34 KB	Ollama HTTP client for the text and vision models. Holds the planning system prompt (natural language to JSON action plan), JSON extraction helpers, health checks, and a smart rule-based natural-language step parser as LLM fallback.
agents/orchestrator.py	~25 KB	Coordinates an execution end-to-end: .env hot-reload per run, \${VAR} substitution (flow env > os.environ > .env), captured-value substitution, cancellation tracking, artifact storage, failure memory recording, and PDF report generation.
agents/pdf_reporter.py	~13 KB	Generates per-execution PDF test reports with ReportLab.
agents/memory_store.py	~8 KB	Per-flow persistent memory: records failures and exposes list/load APIs (surfaced via /api/memory endpoints).

4.2 API Routes (backend/app/api/routes)

Route module	Purpose
auth.py	Platform login; issues JWT tokens (admin credentials from env, default admin/admin123)
flows.py	CRUD for flows stored in the artifact store
disk_flows.py	Manages JSON flow files stored on disk under the flows/ directory (DISK_FLOWS_DIR)
execution.py	Start/monitor/cancel executions; WebSocket streaming of logs and browser frames
bulk.py	Bulk execution of multiple flows; sequential runs with shared browser session; bulk results

Additional app-level endpoints in main.py: GET / (service info), GET /health (LLM, vision, and Chromium health with fix hints), GET /api/memory and /api/memory/{flow_name} (flow memory), plus a static mount at /screenshots.

4.3 Core, Models, and Storage

Module	Purpose
core/config.py	Settings singleton: app metadata, JWT secret, admin credentials, Ollama host/models/timeouts, browser options (headless, slow-mo, navigation timeout), screenshot mode, artifact directory layout, CORS origins. Creates artifact directories on startup.
core/security.py	Password hashing and JWT creation/validation helpers
models/schemas.py	Pydantic models: ExecutionStatus, ExecutionLog, ExecutionResult, FlowResponse, etc.
storage/artifact_store.py	File-based persistence for executions, flows, reports under backend/artifacts/
api/deps.py	FastAPI dependency: get_current_user (JWT-protected routes)

5. Frontend Application (frontend/)

Next.js App Router project written in TypeScript with Tailwind CSS and Radix UI primitives. Pages map directly onto platform features:

Route / File	Feature
app/(auth)/login	Login screen (JWT auth against the backend)
app/dashboard	Overview dashboard
app/flows, flows/new, flows/edit	Flow listing, creation, and editing (natural-language step editor)
app/execution, execution/[id]	Execution launcher and live execution detail view (largest page, ~19 KB) with WebSocket-driven live logs and browser frames
app/bulk, bulk/[id], bulk/results	Bulk run setup, per-bulk-run detail, and aggregated results
app/logs/[id]	Execution log viewer
app/reports	Generated PDF report listing/download
app/env	Environment variable management UI (flow credentials, URLs)
app/settings	Platform settings
components/layout	Header and sidebar navigation
hooks/useWebSocket.ts	WebSocket subscription hook for live execution data
lib/api.ts, types/index.ts	Axios API client and shared TypeScript types

Environment files: .env.development.local (local API base URL) and .env.production. next.config.js holds build configuration.

6. Test Flow Definitions (flows/json-flow)

Disk-based flows are JSON files with a name and an ordered list of natural-language steps. Three example flows ship with the project, targeting telecom CRM/inventory systems (ICC inventory and the Tusass self-care/CRM environment):

Flow file	What it does
New_Account_creation.json	50-step flow: log into an inventory system, search available ICCIDs, copy one, log into a second system (MSITCOS), find a customer account, and create a mobile Tusass service with the captured ICC number; takes screenshots (SS steps) before and after submission.
Tusass-Extra-data-add.json	Adds an extra data package to a Tusass subscription.
Tusass-TopUp-Talk.json	Performs a talk-time top-up flow.

Representative step syntax from New_Account_creation.json:

```
"Navigate to ${ICC_URL}"
"Enter Username in input box ${CSR_USER}"
"Click the - Log in - button"
"In the External Id column copy one ICCID"
"Click xpath //*[@id='button-create-service']"
"Wait 2000ms" / "SS" (screenshot)
```

Steps mix free-form English, explicit XPath targeting, timed waits, screenshot markers, and \${VAR} placeholders resolved from the environment at run time - demonstrating the hybrid LLM-plus-deterministic execution model.

7. Folder Structure

```
QA-Automation-Agent/
+-- README.md           Setup and run instructions
+-- start-local.ps1     Windows dev launcher (backend :8000, frontend :3001)
+-- start-production.sh Production startup script (Linux)
+-- backend/
|   +-- requirements.txt Python dependencies
|   +-- venv/            Committed Python 3.13 virtual environment
|   +-- app/
|       +-- main.py      FastAPI app, routers, /health, /screenshots mount
|       +-- agents/      browser_agent, llm_client, orchestrator,
|           |            pdf_reporter, memory_store
|       +-- api/routes/   auth, flows, disk_flows, execution, bulk
|       +-- core/        config (Settings), security (JWT/bcrypt)
|       +-- models/      Pydantic schemas
|       +-- storage/     artifact_store (file-based persistence)
+-- frontend/
|   +-- package.json     Next.js 15 / React 19 / Tailwind / Radix
|   +-- app/             Pages: login, dashboard, flows, execution,
|       |               bulk, logs, reports, env, settings
|       +-- components/layout/ header, sidebar
|       +-- hooks/ lib/ types/ useWebSocket, axios API client, TS types
+-- flows/json-flow/     Disk flow definitions (3 JSON flows + image)
+-- tools/               (empty)
+-- sample-image/        (empty)
+-- .claude/             Claude Code local settings
```

8. Configuration and Environment Variables

Variable	Default	Purpose
OLLAMA_HOST	http://localhost:11434	Ollama API endpoint
LLM_MODEL	qwen2.5:14b	Text model for flow planning
VISION_MODEL	qwen2.5vl:latest	Vision model for screen understanding
LLM_TIMEOUT	120	LLM request timeout (seconds)
BROWSER_HEADLESS	true	Run Chromium without a visible window
BROWSER_SLOW_MO	0	Slow-motion delay between actions (ms)
BROWSER_NAVIGATION_TIMEOUT	60000	Page navigation timeout (ms)
SCREENSHOT_MODE	fail_only	fail_only final all
ADMIN_USERNAME / ADMIN_PASSWORD	admin / admin123	Platform login credentials
SECRET_KEY	(insecure default)	JWT signing key
ALLOWED_ORIGINS	localhost:3000/3001	CORS allow-list
DISK_FLOWS_DIR	/flows	Location of on-disk JSON flows

A notable design choice: the orchestrator hot-reloads the .env file before every execution, so changes to credentials, URLs, screenshot mode, or headless setting apply to the next run without restarting the server. Flow-level env values take priority over process env and .env.

9. Observations and Recommendations

9.1 Strengths

- Fully local AI stack - no data leaves the machine, well suited for testing internal telecom systems.
- Resilient design: rule-based parser fallback when the LLM is down, Chromium relaunch recovery, login-session reuse in bulk runs, and per-flow failure memory.
- Good operational ergonomics: /health endpoint with actionable fix hints, .env hot-reload, configurable screenshot modes to trade speed vs. evidence.
- Clean separation of concerns: agents (engine), routes (API), storage (artifacts), frontend (UI).

9.2 Recommendations

- **Security hardening:** default SECRET_KEY ("super-secret-key-change-in-production") and admin/admin123 credentials should be overridden via .env before any shared deployment.
- **Model documentation mismatch:** the README instructs pulling qwen2.5:0.5b and moondream, but config.py defaults to qwen2.5:14b and qwen2.5vl. Align the README with the actual defaults (or vice versa) to avoid setup confusion.
- **Repository hygiene:** backend/venv (a full Python 3.13 virtual environment) and __pycache__ directories live inside the project folder; ensure they are git-ignored to keep the repository lean. frontend/tsconfig.tsbuildinfo is also a build artifact.
- **browser_agent.py size:** at ~130 KB / single module, the executor would benefit from being split (action handlers, screenshot logic, Windows loop management) for maintainability.
- **Tests:** pytest and pytest-asyncio are declared but no test files were found in the project tree - adding unit tests around step parsing and \${VAR} substitution would be high-value.

- **Empty folders:** tools/ and sample-image/ are empty; remove or populate them.

10. Appendix

10.1 Python Dependencies (backend/requirements.txt)

Package	Version
fastapi	0.115.6
uvicorn[standard]	0.32.1
python-jose[cryptography]	3.3.0
passlib[bcrypt]	1.7.4
httpx	0.28.1
playwright	1.49.1
pydantic	2.10.4
python-multipart	0.0.20
python-dotenv	1.0.1
aiofiles	24.1.0
pytest	8.3.4
pytest-asyncio	0.25.0
reportlab	>=4.2.0

10.2 Key NPM Dependencies (frontend/package.json)

Package	Version
next	15.1.3
react / react-dom	^19.0.0
typescript	^5.7.3
tailwindcss	^3.4.17
axios	^1.7.9
lucide-react	^0.469.0
@radix-ui/react-* (dialog, tabs, tooltip, dropdown-menu, scroll-area, separator)	^1.x / ^2.x
class-variance-authority / clsx / tailwind-merge	^0.7.1 / ^2.1.1 / ^2.6.0
date-fns	^4.1.0

10.3 Recent Repository Activity

Commit	Message
d45dd80	feat: bulk result page
5fa949f / 509a1f2	fix: flow disk
0e73ad8	fix: production env error

Commit	Message
ead3aef	fix: typescript code error in dependencies
e4aba42	feat: data add in flow and report
e937224	fix: LLM speed and agent change with server side
d697597	feat: add end-to-end project

Data source: static analysis of the project tree at C:\Users\Sandhosh\Downloads\QA-Automation-Agent on 2026-06-12.
Generated with ReportLab.